Identity, Permissions, and Row Security

A useful report can omit private fields.
A valid login can still be denied a row.

Day 1: a report reader with limited permissions
Day 2: two residents with different visible tickets

Changing the requested ID must not reveal another person

Resident A
Signed in correctly



```
GET /tickets/3
```

Ticket	Owner	Should Resident A read it?
1	Resident A	Yes
3	Resident B	No

The server must enforce ownership even when the caller chooses a different ID.

Security terms answer different questions

Term	Meaning in this request
Identity	The account claiming to be Resident A
Authentication	Verification of the login or token
Authorization	The decision to allow this action on this data
Auditing	Records of relevant actions and decisions

A working password does not grant every database privilege.

The report reader needs a narrow interface

Actor	Required work	Outside the task
Report reader	Read open ticket IDs, categories, and statuses	Read resident email or change tickets
Lab analyst	Read ticket summaries and count by status	Read private source tables or edit requests

Least privilege starts with the work the role must support. Then we test what remains unavailable.

A role can collect privileges without having a password

```
CREATE ROLE report_reader_demo NOLOGIN;
GRANT report_reader_demo TO CURRENT_USER;
```

```
BEGIN;
SET LOCAL ROLE report_reader_demo;
SELECT session_user, current_user;
ROLLBACK;
```

session_user: original database login

current_user: effective role for this test

Use a personal practice project. Membership permits SET ROLE even when the group role has NOLOGIN.

The complete reporting interface

```
CREATE SCHEMA security_demo;
CREATE VIEW security_demo.open_ticket_report AS
SELECT ticket_id, category, status
FROM metro_support.tickets
WHERE status = 'open';

GRANT USAGE ON SCHEMA security_demo, metro_support
TO report_reader_demo;
GRANT SELECT ON security_demo.open_ticket_report
TO report_reader_demo;
```

Load the Metro Support fixture first. The role receives access to the view, not to the underlying tickets table.

An allowed test names the role and the returned records

```
BEGIN;
SET LOCAL ROLE report_reader_demo;
SELECT current_user, ticket_id, status
FROM security_demo.open_ticket_report ORDER BY ticket_id;
ROLLBACK;
```

Effective role	Ticket IDs	Status
report_reader_demo	1001, 1007, 1011	open for all three

Run the entire actor/query/rollback batch on one connection. The lab supplies a Colab display option.

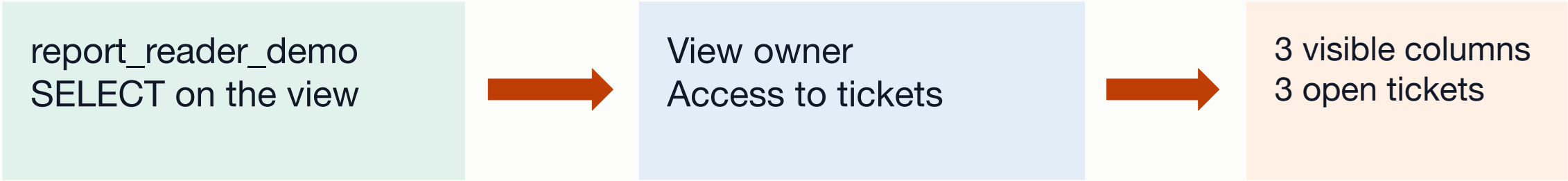
A denied test remains safe if the grant is wrong

```
BEGIN;
SET LOCAL ROLE report_reader_demo;
UPDATE metro_support.tickets
SET priority = 'high' WHERE ticket_id = 1004;
ROLLBACK;
```

Expected: permission error, SQLSTATE 42501.
After an error, run ROLLBACK separately if execution stopped.

If the UPDATE unexpectedly succeeds, rollback still protects the row. Fix the grant and repeat the test.

View access and base-table access can differ



A normal view can use its owner's underlying-table permissions. The reader does not need a broad grant on the source.

An elevated-owner view is not automatically a resident-safe RLS interface. PostgreSQL 15+ supports security_invoker views for caller-based checks.

Different failures call for different checks

Observed result	First distinction to investigate
Network unreachable	Can the client reach the endpoint? No role test occurred.
Password authentication failed	Did the intended database login authenticate?
SQLSTATE 42501	Which role, object, and operation were rejected?
Zero rows, no error	Did the query or a row policy exclude the records?

Broad grants do not repair routing or credentials. Check the failing layer first.

Lab 1: a useful report under a restricted role

Create the supplied analyst role and limited view.

Read the view, then write a count-by-status report as that role.

Test the denied email read and ticket update.

Explain what the analyst can do and what the tests establish. Keep the SQL and explanation together.

Brightspace: week_06_least_privilege.sql. Run the lab cleanup after testing.

Day 2: the same query, different rows

Both residents may read the table.
Each resident should see only their own tickets.

We will test the ownership rule, a direct lookup, and a new row.

Four rows make the ownership rule visible

ticket_id	owner_role	subject
1	resident_a_demo	Broken bench
2	resident_a_demo	Dark streetlight
3	resident_b_demo	Missed pickup
4	resident_b_demo	Leaking hydrant

security_demo.resident_tickets starts with these four rows. Both resident roles have schema USAGE and table SELECT, but no write grant.

A policy compares each row with the effective role

```
ALTER TABLE security_demo.resident_tickets
ENABLE ROW LEVEL SECURITY;

CREATE POLICY read_own_demo
ON security_demo.resident_tickets
FOR SELECT TO resident_a_demo, resident_b_demo
USING (owner_role = current_user);
```

USING keeps rows whose ownership comparison is true. The caller's SELECT does not need to include that comparison.

With RLS enabled but no applicable policy, an ordinary reader sees no rows.

The same SELECT demonstrates different visibility

```
BEGIN;
SET LOCAL ROLE resident_a_demo;
SELECT current_user, ticket_id, subject
FROM security_demo.resident_tickets ORDER BY ticket_id;
ROLLBACK;
```

Change only the tested role	Expected IDs
resident_a_demo	1, 2
resident_b_demo	3, 4

The test uses one complete batch. Observe current_user, not just the account shown in the browser.

A direct lookup does not bypass the ownership rule

```
BEGIN;
SET LOCAL ROLE resident_a_demo;
SELECT ticket_id, subject FROM security_demo.resident_tickets
WHERE ticket_id = 3;
ROLLBACK;
```

Expected result: zero rows. No permission error.

The table read is allowed. Ticket 3 does not satisfy the policy for Resident A, even when the query names it directly.

The rule must continue working when data changes

```
INSERT INTO security_demo.resident_tickets
  (ticket_id, owner_role, subject)
VALUES (5, 'resident_a_demo', 'Blocked drain');
```

Role	Before the insertion	After the insertion
resident_a_demo	1, 2	1, 2, 5
resident_b_demo	3, 4	3, 4

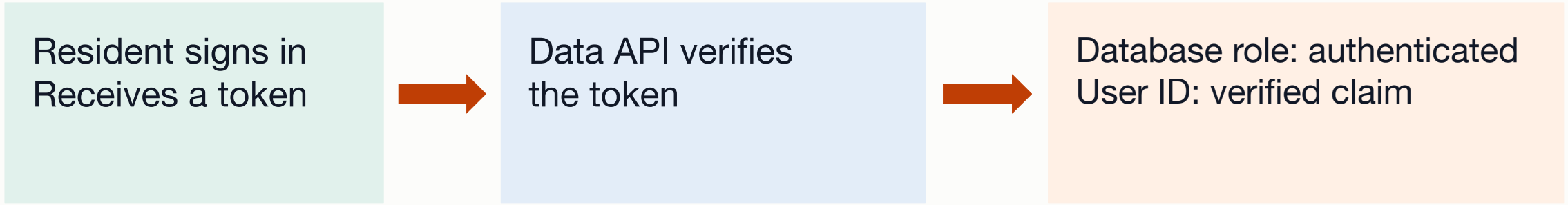
Insert as the administrator, then rerun the resident tests. The policy is unchanged.

The administrator is a different test actor

Query context	Visible IDs after the demo insertion
Ordinary Resident A role	1, 2, 5
Ordinary Resident B role	3, 4
Administrative owner with bypass	1, 2, 3, 4, 5

Owners normally bypass RLS. Superusers and BYPASSRLS roles bypass it. FORCE ROW LEVEL SECURITY does not remove superuser or BYPASSRLS powers.

Supabase separates the application user from the database role



Many people share the authenticated role. auth.uid() identifies the user for the current verified request.

The browser cannot establish ownership merely by sending someone else's user ID. A signed token is not necessarily encrypted.

A real ownership column must match verified user identities

```
CREATE POLICY residents_read_own_tickets
ON metro_support.tickets
FOR SELECT TO authenticated
USING (requester_auth_id = (SELECT auth.uid()));
```

Illustration, not setup for today's integer-ID fixture.

Requires a correctly populated UUID ownership column, appropriate grants, RLS enablement, and the intended API route.

If auth.uid() is NULL, the equality is unknown rather than true. It selects no row.

Reading a row and accepting a new row are separate rules

Policy clause	What it checks	Ownership example
USING	Existing rows available to the operation	Resident A reads A's tickets
WITH CHECK	Proposed inserted or updated row values	A new ticket must belong to the acting resident

```
FOR INSERT TO authenticated
WITH CHECK (requester_auth_id = (SELECT auth.uid()))
```

Illustrative policy fragment. INSERT also needs an object grant. Today's lab tests SELECT only.

Different credentials carry different powers

Credential	Appropriate use	Important limit
Publishable client key	Identify the application component	Does not identify one resident
User access token	Present a verified user session	Keep usable tokens private
Secret or service-role key	Trusted server-side component	Elevated access can bypass RLS
Database password or URI	Connect as a database login	Not an application-user token

If a secret leaks, revoke or rotate it first. Removing a line from Git does not invalidate the old value.

Lab 2: test ownership with another row

Run the supplied two-resident case and the direct lookup.

Add ticket 5 for resident_102_lab as the administrator.

Rerun both residents' queries without changing the policy.

In SQL comments, explain the results to the developer and identify the real-token test still needed before deployment.

Brightspace: week_06_rls_test.sql. Complete the cleanup in the lab.

When MVCC goes wrong

On July 20, 2015, PostgreSQL at Sentry
stopped accepting writes.
Nobody attacked it. No disk failed.
It stopped on purpose.

Recap row versions, meet vacuum, see why teams tune it, then read the incident and write.

Recap: an update makes a new row version

```
SELECT xmin, xmax, ticket_id, priority
FROM metro_support.tickets WHERE ticket_id = 1004;
```

Version of ticket 1004	xmin	xmax	Still visible to
medium, open	733	735	snapshots older than 735; then dead
high, open	735	0	every later snapshot

Every row version carries the transaction ID that created it and the one that expired it. Once no snapshot can see the old version, it is dead, and nothing in UPDATE or DELETE ever removes it.

Transaction IDs are a counter with a horizon

A transaction ID is a 32-bit number on a circle. From any point, about two billion IDs are the past and two billion are the future.

If the counter laps a row, its creating transaction looks like the future and the row vanishes. An old delete looks undone, and the deleted row comes back. Nothing on disk changed.

Freezing marks a row visible to everyone forever, so its age stops mattering. VACUUM freezes rows older than vacuum_freeze_min_age (50 million).

PostgreSQL records the oldest unfrozen ID per table (relfrozenxid) and per database (datfrozenxid). Its age is the number to watch.

Vacuum is the maintenance that keeps MVCC honest

What VACUUM does	Why it matters
Removes dead row versions; marks the space reusable	Tables and indexes stop growing from old versions
Freezes old transaction IDs	The counter can keep going without lapping old rows
Maintains the visibility map; ANALYZE refreshes statistics	Index-only scans and good plans depend on them

Plain VACUUM runs alongside normal reads and writes and does not shrink the file. VACUUM FULL rewrites the table under an exclusive lock. Neither one means something is broken.

Verified today on a 12-row table: 7 dead rows before VACUUM, 0 after.

Autovacuum decides when to run, one table at a time

```
-- autovacuum vacuums a table once its dead rows exceed
-- autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor * reltuples
--           50                +           0.2                 * row count
```

Table size (reltuples)	Dead rows before autovacuum starts
12 rows (Metro Support tickets)	53
1,000 rows	250
1,000,000,000 rows	200,000,050

A second trigger ignores dead rows: a table whose oldest unfrozen ID is older than autovacuum_freeze_max_age (200 million) is vacuumed to freeze it, even with autovacuum turned off.

Why teams tune autovacuum, and the trap

Setting	Default	Why someone changes it
autovacuum_vacuum_cost_delay	2 ms	Raise it so vacuum uses less disk I/O at peak
autovacuum_max_workers	3	Raise it when many big tables fall due together
autovacuum_vacuum_scale_factor	0.2	Lower it per table so a huge table is vacuumed sooner
autovacuum_freeze_max_age	200 million	Raise it to delay forced wraparound vacuums
maintenance_work_mem	64 MB	Raise it so one pass remembers more dead rows

Every change that makes vacuum cheaper today moves work to later. The counter does not wait.

Wraparound protection: unavailable rather than wrong

Transactions left before wraparound	What PostgreSQL 16 does
More than 40 million	Normal operation. Forced freezing vacuums begin at age 200 million.
40 million	WARNING: database "mydb" must be vacuumed within N transactions
3 million	ERROR: database is not accepting commands to avoid wraparound data loss

Reads still work. Every INSERT, UPDATE, DELETE, and TRUNCATE fails until a VACUUM in that database freezes the old rows. Sentry hit the 2015 version of this limit, drawn at one million.

The error's hint still mentions single-user mode. Current documentation says to end old transactions, drop stale replication slots, and run VACUUM normally.

Sentry, July 20, 2015: the day the counter ran out

What happened	What it meant
Writes stop; the database is effectively read-only	Wraparound protection fired. The event queue starts backing up.
Running vacuums left to finish; primary moved to bigger hardware	Nearly three hours with nothing to show for it
Every table done except one enormous event-to-rollup map	No way to know how many more hours it needs
That table is truncated	Non-critical features; replicas hold a copy. Service back five minutes later.

Afterward: six workers, a 15-second naptime, zero cost delay, far more maintenance memory, and a plan to split the data across databases.

The people who ran TRUNCATE held the most powerful credentials in the system.

Two queries that tell you how close you are

```
SELECT datname, age(datfrozenxid) AS xid_age
FROM pg_database ORDER BY xid_age DESC;

SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 5;
```

Reading	Healthy	Worrying
Database transaction ID age	Well under 200 million	Climbing toward 2 billion
Dead rows per table	Falls back after each vacuum	Only grows; last_autovacuum empty or days old

My practice server today: age 14 for every database; 7 dead rows before VACUUM, 0 after.

Write: when MVCC goes wrong

Read the Sentry incident report (10 minutes).

Write 250 to 350 words in the Brightspace assignment (20 minutes):
1. What PostgreSQL was protecting, and why stopping was right.
2. What let the maintenance fall behind, and why that is a correctness problem.
3. The decision to truncate a table: acceptable? Would you have made it?
4. Two concrete prevention steps, and which matters more.

Use three of the four terms correctly: transaction ID, MVCC, vacuum or autovacuum, wraparound protection. Keep availability and correctness apart.

Brightspace: Week 6 writing response, text only. No SQL, no screenshots. Ground every claim in the article.

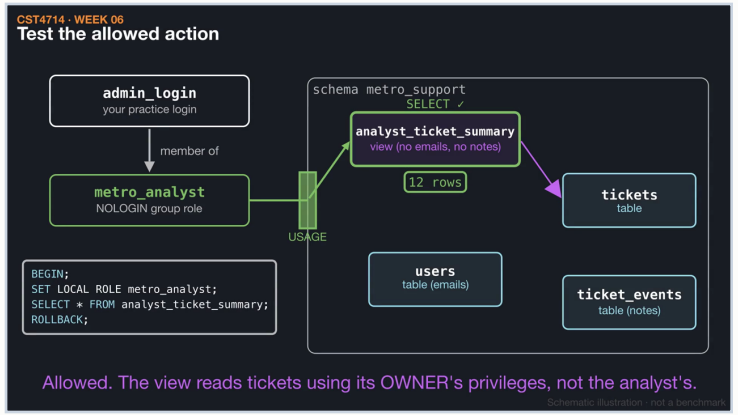
A security test supports a specific claim

The analyst can produce the report.
Each resident sees the correct tickets.
The checks run as the intended role.

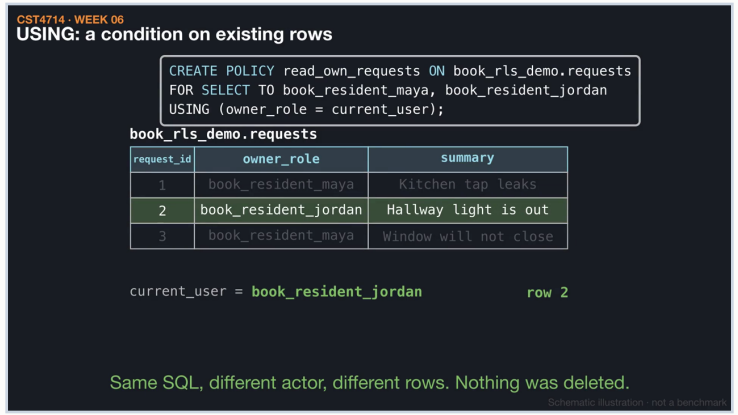
Those results support the tested database rules. They do not establish that every API route, token, or privileged credential is safe.

Next: query plans explain the work behind a permitted query.

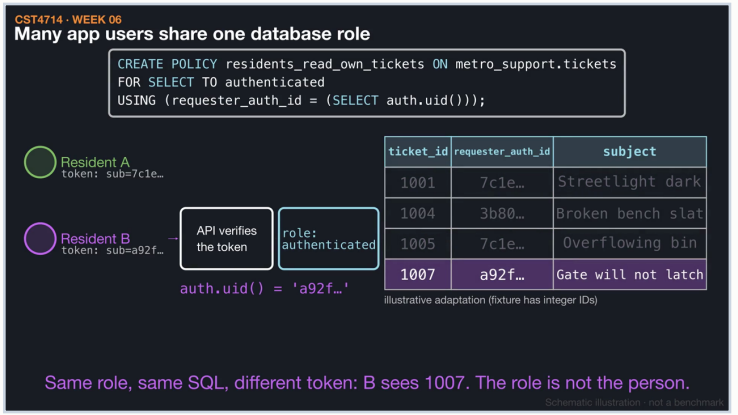
This week's videos



Least privilege: roles and grants (1:24)



Row-level security: USING and WITH CHECK (1:30)



Who is asking? Gates, roles, and tokens (1:33)

Click a thumbnail to play on GitHub. Silent, with captions and .srt subtitles; diagrams are schematic.